DEEP DOUBLE DESCENT: A REDUCED REPRODUCTION

Group 15: Hans Peter Raaschou-Jensen, Harrald Franck, Mathias Elmegaard og Julian Barragan

Assignment 2: Replication Track
`https://github.com/HP2706/ATDL-Group-Work`

ABSTRACT

We replicate a subset of the experiments in "Deep Double Descent: Where Bigger Models and More Data Hurt" by Nakkiran et al. Using the authors' released ResNet18 and five-layer CNN definitions inside our own PyTorch training pipeline, we reproduce width sweeps, training trajectories and sample-size comparisons on CIFAR-10 and CIFAR-100, and compare them with the authors' released measurements. We further re-implement the IWSLT'14 Transformer experiments. To fit our budget, we train for at most a tenth of the original horizon with one seed per setting (229 vision runs, 14.98 B200 GPU-hours). Model-wise double descent under label noise, and the shift of the interpolation threshold with noise, augmentation and sample size, are reproduced qualitatively. Epoch-wise double descent is replicated for sufficiently large models, while small and medium sized models follow a monotone and U-shaped test error trajectories. The CNN plateau, where more data does not help, does not appear at the authors' widths, which we attribute to the fixed update budget. The noise-free peaks reported for CIFAR-100 are not confirmed at our horizons. [Translation: one sentence missing??]

1. INTRODUCTION

Classical learning theory predicts a U-shaped bias-variance trade-off, where larger models overfit beyond some complexity. Modern networks contradict this, generalising better as they grow far beyond the number of training samples. Which regime applies matters in practice. It affects whether to enlarge a model, train it longer, stop early or collect more data.

Nakkiran et al. [1] reconcile the two views through the *effective model complexity* (EMC) of a training procedure: the largest number of samples it can fit to near-zero training error (Section 2). They hypothesize that test error behaves atypically when the EMC is close to the number of training samples n. Varying the width, the training time or n moves a model through this critical regime. The results are *model-wise* and *epoch-wise* double descent, and *sample-wise* non-monotonicity, where more data can hurt. The authors demonstrate these effects for CNNs, ResNets and Transformers on CIFAR-10/100 and machine translation. We test five of their claims:

- C1 Model-wise double descent occurs with the test-error peak at the interpolation threshold.
- C2 Label noise, data augmentation and more training samples move this threshold, and the peak, to larger models.
- C3 Small, intermediate and large models show respectively monotone, U-shaped and double-descent test-error trajectories over training time.
- C4 Near the threshold, more training data does not help and can increase test error.
- C5 Optimal early stopping largely removes these peaks

Our contributions are threefold. First, we wrote an independent PyTorch pipeline for training, evaluation and checkpointing around the authors' released model definitions. Second, we replicate the model-, epoch- and sample-wise vision experiments on CIFAR-10/100 and compare them directly with the authors' released measurements. We also re-implement the IWSLT'14 Transformer experiments of their Figures 3 and 11(b). Third, we report the compute cost of this reduced protocol. In summary, C1 and C2 are reproduced qualitatively, C3 and C4 only partially, and our results are consistent with C5. The noise-free peaks reported for CIFAR-100 remain unconfirmed at our training horizons.

2. BACKGROUND

A training procedure $\mathcal{T}$ maps a training set $S = \{(x_i, y_i)\}_{i=1}^{n}$ to a classifier $\mathcal{T}(S)$. It covers architecture, optimizer, learning-rate schedule, number of steps and augmentation. For a data distribution $\mathcal{D}$ and tolerance $\epsilon > 0$, the EMC is

$$\mathrm{EMC}_{\mathcal{D},\epsilon}(\mathcal{T}) = \max\{n \mid \mathbb{E}_{S\sim\mathcal{D}^n}[\mathrm{Error}_S(\mathcal{T}(S))] \le \epsilon\}, \tag{1}$$

where Error_S is the error on the training set itself and the authors use $\epsilon = 0.1$. Their generalized double-descent hypothesis distinguishes three regimes. If $\mathrm{EMC} \ll n$ (under-parameterized) or $\mathrm{EMC} \gg n$ (over-parameterized), any change that increases the EMC decreases test error. If $\mathrm{EMC} \approx n$ (critically parameterized), such a change may either increase or decrease it. The *interpolation threshold*

is the point where $\mathrm{EMC} = n$. Since the EMC cannot be computed directly, we follow the authors and estimate the threshold as the smallest width $\hat{k}_{\mathrm{interp}}$ whose final training error on the (possibly noisy) training labels is at most ϵ.
Increasing width or training time raises the EMC and moves a model through the threshold. Increasing n, adding label noise or adding augmentation makes the training set harder to fit and shifts the threshold to larger models. Model-, epoch- and sample-wise double descent (C1, C3, C4) are thus one-dimensional slices through the same picture, and C2 describes how that picture shifts.

3. METHODOLOGY AND EXPERIMENTAL DESIGN

3.1. Vision setup

3.1.1. Reduced Training

The authors publish extensive training and evaluation logs. This allowed us to understand how many epochs or optimization steps were needed before their results materialized. We found that plotting their results for the first 10% of epochs we were able to get results that looked almost visually identical to what they reported. Thus to cover several experimental settings within our compute budget, we reduced Adam training from 4,000 to 400 epochs and CIFAR-10 SGD training from 500,000 to 50,000 updates. These are 10% of the original experiment; the CIFAR-100 CNN run instead uses 5% of the stated one-million SGD steps. For all graphs we plot we are using the

3.1.2. Model implementation

The original computer vision experiments used two architecture families: width-scaled ResNet18 and a five-layer CNN. We conduct experiments with both model families reusing the authors' model code for ResNet18 and the five-layer CNN, rather than reimplemting the architectures.
Unofrtunately the authors' released repository did not come with out of the box training code, so we wrote our own PyTorch training code, data loading, evaluation, resumable checkpointing, and concurrency primitives that allow us to manage multiple training runs on the same GPU.
Our loader scales pixels to $[0, 1]$ without channel normalization, samples one fixed corruption mask per seed, and replaces selected labels with uniformly sampled incorrect classes. This noise procedure follows Section 4 of the paper, which specifies that label noise is sampled once rather than at each epoch [1]. Subsets use a seeded permutation, implementing the paper's random subsampling setup. Augmentation, when enabled, uses four-pixel padding, a random 32×32 crop, and horizontal flipping, as specified in Appendix B.2.
the paper and released model and plotting code do not establish whether the authors used channel normalization. So we choose a reasonable default for the fields Scaling pixels by $1/255$ is our default. We cannot verify that it matches the authors' preprocessing, and normalization could affect optimization and the resulting curves.
We evaluate the training subset without augmentation against its fixed training labels and the full 10,000-image test split against clean labels. Mean cross-entropy and misclassification fractions are recorded every ten epochs for Adam and every 1,250 steps for SGD, for comparison with the authors' results.

3.1.3. Training and optimizers

We follow the training protocol from the paper closely. All reported runs use batch size 128 as the paper, cross-entropy loss, seed zero, FP32 training, and no weight decay.
Adam runs use a constant learning rate 10^{-4} with PyTorch's default moments $(\beta_1, \beta_2) = (0.9, 0.999)$ and $\epsilon = 10^{-8}$. SGD uses no momentum and our explicitly implemented schedule

$$\eta_t = \frac{0.1}{\sqrt{1 + \lfloor t/512 \rfloor}},$$

where t is the number of completed updates. Which implements the paper's inverse-square-root learning rate schedule.

3.1.4. Hidden Dimension size

The original paper does a sweep of each dimension from 1-128. In order to save compute we sweep a much smaller grid of $\{2, 3, 4, 6, 8, 12, 16, 24, 32, 64\}$ for Resnet18 and $\{1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 64\}$ for CNN.
For the full-data CNN trained with augmentation we use $\{1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 36, 40, 48, 64, 96, 128\}$.
The reason behind the non uniform grid-selection comes down to certain figures relying on a specific width, that thus has to be added to the grid. Moreover we try to have a higher density for smaller widths, which is the area most intertsing for observing the double descent behaviour.
We include a full comparison of our setup with the paper's stated protocol and released code in Appendix B, Table 5

3.2. Translation setup

3.2.1. Model

We use the same Transformer implementation as the paper, namely the fairseq Transformer Fairseq v0.9.0. Its six-layer encoder and decoder use eight attention heads, feed-forward width $4d_{model}$, sinusoidal positions, and post-normalization.

3.2.2. Experiment selection

For the translation experiments we choose to try to reproduce Figure 3 and Figure 11b in the paper. They both use the

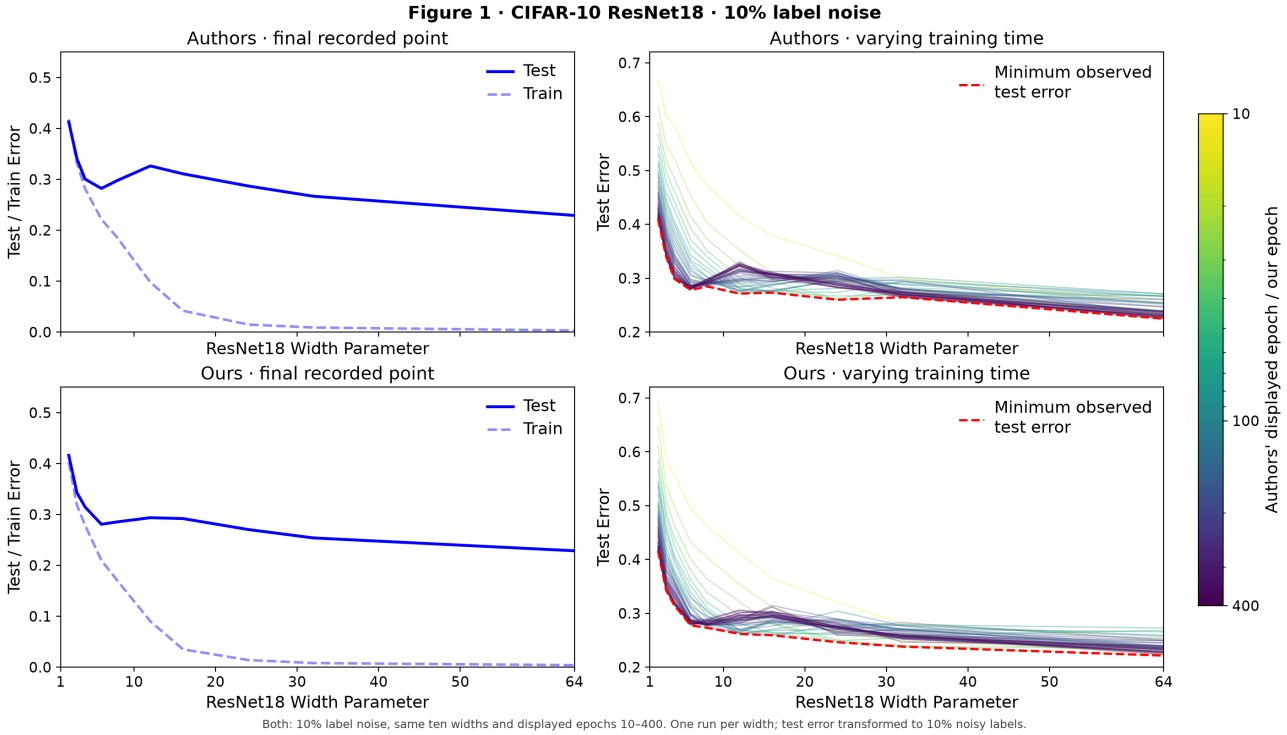


Fig. 1. Noise-matched adaptation of the opening figure. The original opening figure instead uses 15% noise and 4,000 epochs [1]. For this figure we use 10%-noise CIFAR-10 ResNet18 measurements above our 10%-noise results. Both panels use the same ten widths, Results fir 400 epochs, and 10%-noise. Red curves show the minimum observed test error over the displayed evaluations, not validation-based early stopping.

IWSLT'14 German → English dataset and the same transformer family. They contain complimentary experiments: Figure 3 varies d_{model} while comparing 4k and 18k samples, and Figure 11b holds d_{model} fixed at 64 and 80 and varies the sample size. Together they test the claim of "more data can hurt" from two directions, and they also test whether the phenomenons of model-wise double descent and sample-wise non-monotonicity in a different domain.

3.2.3. Experimental setup and reproduction choices

The description of the experimental setup is limited, but the authors describe that they use the IWSLT'14 German → English dataset. They train a 6-layer encoder-decoder Transformer with 8 attention heads. The model size varies in the experiments with d_{model}, and the feed-forward dimension is $d_{ff} = 4d_{model}$. They use 10% label smoothing and no dropout, and they train for 80k gradient updates. The IWSLT text was preprocessed using Byte-Pair-Encodings, and they used the Fairseq sequence-modeling toolkit for the Transformer implementation. Training follows the optimization procedure of Vaswani et al. [2], with a linear warmup phase followed by inverse-square-root learning-rate decay using Adam.

Because the experiments were not fully specified in the report nor in their code repository, so we had to make some choices. We used the optimization procedure of Vaswani et al. [1], using Adam with $(\beta_1, \beta_2) = (0.9, 0.98)$ and $\epsilon = 10^{-9}$. We used 4000 warmup steps, and learning rate $lr = d_{model}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$. So the learning rate increases gradually during the warmup, and then decays proportional to the square root of the step number.

For batches we used a max token budget of 4096, rather than a fixed number of sentences. Batches are then formed from similarly sized sentences using a Fairseq-like length-based batching procedure. Therefore the number of sentences per batch vary with sentence length.

For figure 3 we used 25 widths, evenly spaced from 8 to 200, so $d_{model} = \{8, 16, ..., 192, 200\}$, with 4k and 18k training samples. For figure 11b we used $d_{model} = \{64, 80\}$ and $n = 2k, 4k..., 48k, 50k$. So our setup is Fairseq-like, but we do not know for sure if it is exactly the same as the authors.

3.3. Deviations and compute?

Appendix A summarizes our vision sweeps, translation experiments, and coverage of the paper's figures.
The seven jobs used 14.98 GPU-hours in total.

4. RESULTS

4.1. Model-wise Double Descent

4.1.1. Model size, label noise, and optimization

The width sweeps and associated comparisons are organized by their experimental question rather than by compute cost. Paper Figures 1, 2, 4, and 5 concern width and noise or augmentation; Figures 6 and 7 add optimizer and dataset comparisons. All replicated figure families are presented as comparisons of ours with the authors in Appendix C. At 20% noise, our augmented CIFAR-10 CNN test error rises from 21.74% at width 16 to 28.11% at width 48, then falls to 25.88% at width 128 at step 50,000. This single-seed non-monotonicity does not establish the paper's converged interpolation-threshold result.

4.2. Epoch-wise Double Descent

Nakkiran et al. argue that traning time acts like model size. As traning proceeds the EMC grows, so a fixed model can cross the interpolations threshold during training. For Resnet18 and CIFAR-10 with 20% label noise (their Figure 9), this yields three regimes. A small model ($k = 3$) never reaches the threshold, and its test error decreases monotonically. An intermediate model ($k = 12$) only just reaches it and follows a classical U-curve. A large model ($k = 64$) crosses it early, so its test error falls, rises and falls again, eventually below the first minimum: training longer corrects overfitting. Their Figure 10 shows that the peak grows with label noise and is absent without it.
We summarize each trajectory by its first minimum $E(t_1)$, the subsequent peak $E(t_p)$ and the final error $E(T)$. The weak form of the large-model claim requires $E(T) < E(t_p)$, i.e. a second descent. The strong form requires $E(T) < E(t_1)$. All three regimes appear within our 400 epochs (Table 1, Figure 2). The $k = 3$ error decreases monotonically to 28.9%. The $k = 12$ error reaches 20.7% at epoch 80 and then rises to 29.7% at epoch 400, still increasing. The $k = 64$ error falls to 15.1% at epoch 20, peaks at 26.1% at epoch 70 and descends to 20.9% at epoch 400. Finally, the $k = 64$ peak falls in the epoch range of the paper's ResNet peaks. Thus, the weak form holds. However, the strong form does not hold as the final error is still 5.8 points above the first minimum.
The authors train for 4,000 epochs. Their released $k = 64$ trajectory is at 21.9% at epoch 400 and reaches 19.6% at epoch 4,000. The second descent therefore continues well beyond our horizon, and the missing strong form is not supported as

k	Paper	$E(t_1)$	$E(t_p)$	$E(T)$	A: 400 / 4,000
3	monotone	–	–	28.9	29.6 / 24.6
12	U-curve	20.7 (80)	29.7 (400)	29.7	34.1 / 34.6
64	DD	15.1 (20)	26.1 (70)	20.9	21.9 / 19.6

Table 1. Epoch-wise test error (%), CIFAR-10 ResNet18, 20% label noise, one seed. $E(t_1)$: first minimum; $E(t_p)$: subsequent peak (epoch in parentheses); $E(T)$: ours at epoch 400. A: authors' released run at epochs 400 and 4,000.

20% label noise in both ours and the authors experiments. The papers only evidence of the strong form comes from the 10% runs (Fig. 10 in the paper). C3 is therefore partially reproduced. The three regimes and the second descent are reproduced, while *"training longer corrects overfitting"* is shown in its weak from for 20% label noise, while the strong form for 10% label noise is untested rather than contradicted.
The width-128 CNN arm of Figure 10 is shown in Appendix 11 for 0% and 20% label noise, which verifies the authors additional claim that the epoch-wise double descent phenomenon holds for different architectures in the presence of label noise. While without label noise, no epoch-wise double descent occurs.

4.3. Sample-wise, CNN

Paper Figures 11(a) and 12 motivate the sample-size comparisons. In [1] they argue that more samples have two effects on the test error; it falls overall, but the peak moves to larger widths, since fitting more samples requires a larger model. For CNNs they only claim that the two effects cancel near the threshold, such that $4\times$ more samples does not help, whereas more data hurting is shown for Transformers.
The train error is what ties the peak to the sample size. Taking the interpolation threshold as the smallest width with train error at most 0.1, the ϵ of their Definition 2.1, the threshold grows with the sample size in both the authors' grid and ours, and the test-error peak lies close to it (Table 2). The moving peak is thus reproduced (C2).
The plateau (C4) is not, at least not at the same widths. From 12,500 to 50,000 samples the authors' test error falls by 1.9 points at width 32, whereas ours falls by 10.1, and by 3.3 at width 64, next to our 50,000-sample threshold. With a fixed budget of 50,000 updates the three sets get 512, 256 and 128 epochs, so the largest set is the furthest from fitting; its threshold therefore lies further right than the authors', and the plateau, if present, moves with it. Our 12,500 and 25,000 grids have no widths between 32 and 64, so where the curves come closest is not measured. Figure 12 and Figure 14 show the curves.
Appendix B.4 states 500K epochs for Figure 11(a), where Section 4 states 500K gradient steps; read as steps, the authors' sets get 5,120 to 1,280 epochs, ten times ours.

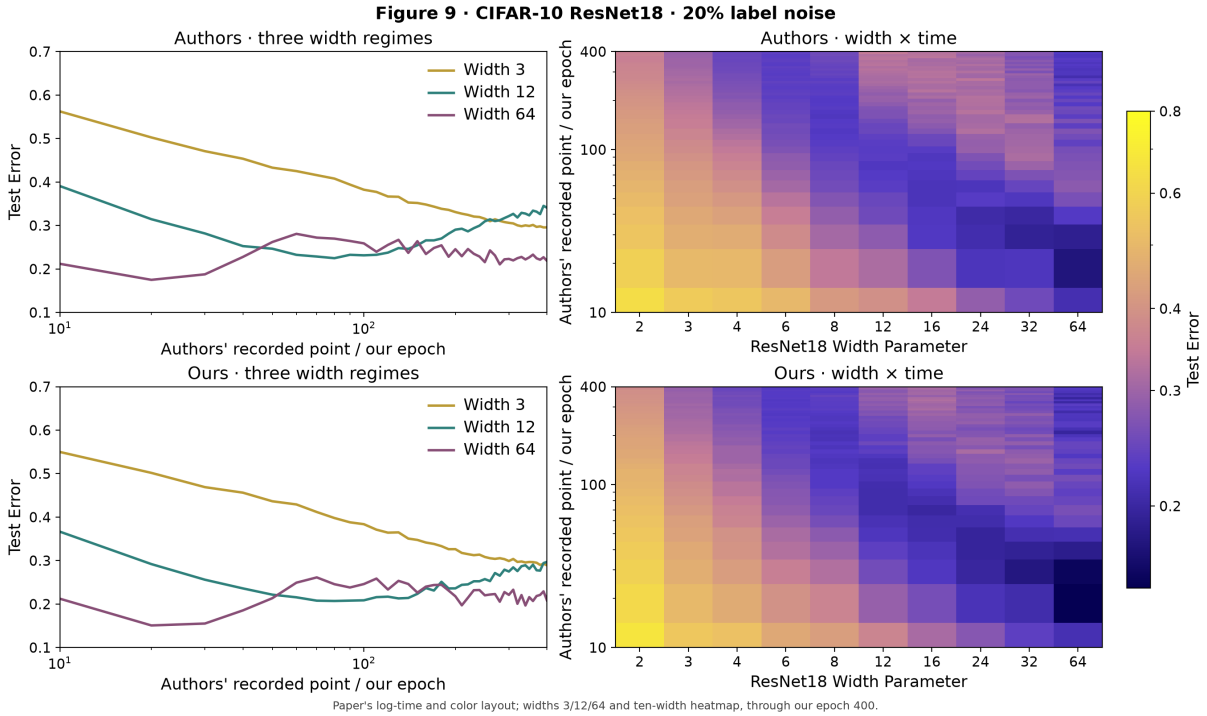


Fig. 2. Test error over training for ResNet18 widths $k = 3, 12, 64$ on CIFAR-10 with 20% label noise: authors' released runs (top, 4,000 epochs) and ours (bottom, 400 Adam epochs, one seed). Authors' epochs follow their notebook convention (measurement index plus one); their logging cadence is undocumented, so the time axes are aligned only approximately.

n	Threshold A / O	Peak A / O	Test, $k = 32$ A / O	Test, $k = 64$ A / O
12,500	15 / 17	15 / 16	33.9 / 35.7	29.7 / 31.2
25,000	23 / 27	20 / 32	34.1 / 34.1	27.0 / 29.0
50,000	34 / 56	31 / 48	32.0 / 25.6	27.0 / 27.9

Table 2. Sample-wise comparison, CIFAR-10 CNN with 20% label noise; authors (A) and ours (O). Threshold: width where train error reaches 0.1, linearly interpolated. Peak: test-error maximum after the first minimum. Test error in % at the common widths either side of the authors' 50,000-sample threshold. Authors on their released 42-width grid, ours on the widths of Table 3.

4.4. Translation results

4.4.1. Model-wise double descent

In the original paper, figure 3 showed that for both 4k and 18k samples, test-loss initially increased with d_{model}, but after peaking around $d_{model} = 24$ for 4k samples, and $d_{model} = 56$ for 18k samples, the test loss decreased with increased dimension continuing decreasing even at $d_{model} = 200$, which fits with the authors broader claim that increasing sample size can shift the critical region towards larger models. The peak in test loss is also observed in our results, but earlier for both 4k and 18k samples. We were not able to reproduce their result where more data can actually hurt performance, as the 18k-sample curve remains below the 4k-sample curve across all tested dimensions, as seen in Figure 5. So we partly reproduce the results of model-wise double descent, but we do not reproduce the result of more data hurting performance.

4.4.2. Sample-wise non-monotonicity

In Figure 11(b), the authors plot the result of an experiment where they instead vary the number of samples, from 2k to 50k, for two models with dimensions 64 and 80. The results in the paper show a non-monotonic region, where increasing the number of samples actually causes worse performance, and then continuing to increase samples beyond that point gives a monotone decrease in the loss. This result supports the authors claim that near the threshold, more training data does not help and can in fact hurt.

Our attempt to reproduce these results were partly successful, and we see that for our $d_{model} = 80$ model, the test loss rises from 4k samples to 6k samples, and we need 12k samples before the loss is lower than at 4k. The $d_{model} = 64$ model shows a weaker initial increase in loss from 2k to 4k samples. So we qualitatively partly reproduce the sample-wise non-monotonicity, but our results differ from the paper in the exact positions of the peaks, and the absolute losses also do

not match. The results are shown in Figure 13.

5. DISCUSSION AND LIMITATIONS

5.1. Ambiguities in the paper and released artefacts

One clear limitation in terms of reproducibility is the lack of a well-maintained codebase. The authors release their model definitions and raw measurements, which made the direct comparisons in this report possible, but the lack of a complete training pipeline lead to some general ambiguities which affected reproducibility.

5.1.1. Translation experiments

The high-level transformer setup was documented, but the exact training procedure was not recoverable. We therefore had to make some guesses and choices of our own. The authors stated that they use a Fairseq style setup, but it is not clear exactly which version or configuration they use. We were also unable to obtain their exact preprocessed files, so the preprocessing may be slightly different, although our data is processed using BPE as they also describe. Is is also unknown exactly which subsets of the data and seeds they used. There was inconsistency in the information about the optimizer between the paper and the authors' codebase. The paper mentions Adam and following the optimization procedure of Vaswani et al. which uses Adam, but the codebase specifies that the translation experiments used SGD. Here we chose to treat the paper as authoritative. It was also confusing exactly which metric they plot, as they write per-token perplexity in the caption of figure 3, but on the actual plot they write Cross-Entropy Test Loss. Since perplexity is the exponential of the cross-entropy, it seems like they did in fact plot cross-entropy, which we did as well. The batching strategy turned out to be important for reproducibility. Switching from random greedy batching to a more Fairseq-like length-based batching changed the effective amount of training per update and made the sample-wise non-monotonicity in Figure 11(b) much clearer. Therefore implementation details affected whether the phenomenon appeared, but we can not necessarily say that they single handedly cause the discrepancy between the results.

5.2. Effect of the reduced protocol

Training for at most a tenth of the original horizon lowers the EMC of every procedure we run, so interpolation thresholds appear at larger widths than in the paper. How much this matters depends on the claim. Statements about the existence and ordering of peaks survive. Statements about where curves for different settings meet, or how far a trajectory eventually descends, do not.

In the epoch-wise experiment the reduction truncates the trajectories rather than distorting them. All three regimes, and the $k = 64$ peak at epoch 70, develop within the first 400 epochs. At epoch 400 our test errors lie within one point of the authors' for $k = 3$ and $k = 64$ (Table 1). What the truncation removes is the tail of the second descent, which is slow: the authors' $k = 64$ run improves by only 2.3 points between epochs 400 and 4,000. The horizon also decides which regime a width belongs to, because the EMC grows with training time. At 400 epochs our 20%-noise threshold is $\hat{k}_{\mathrm{interp}} = 16$, so $k = 12$ has not yet interpolated, consistent with its intermediate role. The authors' $k = 12$ error is flat between epochs 400 and 4,000 (34.1 to 34.6%), which indicates that the U-curve is a stable regime rather than a truncated double descent.

In addition, reducing the horizon is not neutral across sample sizes. With a fixed number of updates a larger set is seen fewer times, so the reduction moves its threshold the most; the moving peak survives this, whereas the plateau, which depends on where the curves for different sizes meet, does not stay at the authors' widths. A reduced sample-wise reproduction should therefore give the largest set enough updates to fit, and shorten the smaller sets instead.

5.3. Limitations

All results rest on one seed per setting. Seed variance is expected to be largest near the interpolation threshold, where the paper attributes the peak to a single noise-sensitive interpolating model. Our data are consistent with this: at epoch 400 our errors are within one point of the authors' for $k = 3$ and $k = 64$, but 4.4 points lower for the near-critical $k = 12$. The observed peak heights and the 1–3.5 percentage-point differences in the sample-wise experiments come from single runs, so we cannot determine whether they persist across random seeds. Although the published sample-wise panel averages five trials, we could not identify those trials in the released files.

Further limitations:

- Our width grids resolve peak locations only to the grid spacing, e.g. between widths 8, 12 and 16 for ResNet18.
- We use 10 and 20% label noise instead of the 15% of Figures 1 and 2, and one dataset per domain.
- The early-stopping curves (C5) take the minimum over test evaluations, an oracle that overstates what validation-based stopping would achieve.
- The exact Fairseq configuration, preprocessing and batching of the translation experiments could not be recovered.

Our results should therefore be read as a qualitative rather than an exact numerical reproduction.

6. CONCLUSION

Our reproductions of the vision experiments overall were successful, visually we found that almost all figures closely resembled the ones from the paper as is visible C
Our reproductions of the translation experiments provide partial support for the paper's results. We reproduced the sample-wise non-monotonicity of Figure 11(b) and the model-wise double descent of Figure 3, but not the region where 4k samples outperform 18k samples. More importantly, our experiments showed that missing details about implementation, in this case especially the batching, can affect whether the reported phenomenon appears and can limit exact reproducibility.

7. REPRODUCIBILITY AND CONTRIBUTIONS

The assignment code and result organization are available at the group's GitHub repository. Detailed experimental coverage and deviations are reported in Appendices A and B.

7.1. Individual contribution

Hans Peter Lyngsøe
Wrote the training code for the vision experiments along with scaffolding for running multiple experiments on the same gpu in parallel in order to fully utilize the hardware. Ran and conducted the experiments for the model-wise double descent experiments
Mathias Elmegaard Nielsen: Epoch-wise double descent (paper Figures 9 and 10). Ran and analyzed the CIFAR-10 ResNet18 trajectories for widths 3, 12 and 64 at 20% label noise as well as the width-128 CIFAR-10 CNN trajectories at 0% and 20% label noise (paper figure 10c). Defined the first-minimum, peak and final-error criteria for the three regimes and compared them with the authors' released runs at epochs 400 and 4,000. Built Table 1 and Figure 2. Wrote the Introduction, Background and subsection 4.2, and the epoch-wise parts of the Discussion. Contributed to abstract and general discussion sections.

Julian Barragan: sample-wise non-monotonicity (paper Figures 11(a) and 12). Ran the 44 CIFAR-10 CNN subset runs at 12,500 and 25,000 samples; compared them with the authors' released sample grid; built the comparison figures and Table 2, locating the interpolation threshold and test-error peak for each sample size; traced the shifted plateau to the fixed update budget, which gives the larger sets fewer epochs; found that the released grid holds a single trial whereas the published panel averages five, and that Appendix B.4 and Section 4 of the paper disagree on the training horizon; wrote subsection 4.3, the sample-wise discussion and the revision of C2.

Harald Franck: Initial plan was to try to reproduce the plot of Figure 3 and 11(b) in the paper and investigate the reproducibility of those. Implemented and ran the translation experiments, investigated reproducibility issues, produced comparison plots, and wrote about this process. Also wrote the translation experiments results and discussion.

8. REFERENCES

[1] Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever, "Deep double descent: Where bigger models and more data hurt," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2021, no. 12, pp. 124003, 2021.

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, vol. 30.

A. EXPERIMENTAL COVERAGE

Table 3 summarizes the vision sweeps. The translation experiments use IWSLT'14 German–English: the model-size sweep uses 25 embedding widths $d_{\text{model}} \in \{8, 16, \ldots, 200\}$ at 4,000 and 18,000 training pairs; the sample-size sweep uses $d_{\text{model}} \in \{64, 80\}$ and $n \in \{2{,}000, 4{,}000, \ldots, 50{,}000\}$. Both use 80,000 updates, Adam with warmup and inverse-square-root decay, and a 4,096-token batch budget (Section 3.2). Table 4 links each reproduced experiment to its comparison figure in Appendix C and distinguishes our coverage from the full published protocol. Saved training trajectories supply multiple plots; each plot does not represent a separate set of training runs.

Dataset/model and setting	Widths k	Runs	Hours	Training horizon
CIFAR-10 ResNet18, augmented	2, 3, 4, 6, 8, 12, 16, 24, 32, 64	30	4.53	400 Adam epochs
CIFAR-100 ResNet18, augmented	2, 3, 4, 6, 8, 12, 16, 24, 32, 64	30	4.92	400 Adam epochs
CIFAR-10 CNN, augmented	1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 36, 40, 48, 64, 96, 128	48	1.45	50,000 SGD updates
CIFAR-10 CNN, no augmentation	1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 64	33	0.74	50,000 SGD updates
CIFAR-10 CNN, no augmentation	1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 64	33	2.13	400 Adam epochs
CIFAR-100 CNN, clean, no augmentation	1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 64	11	0.25	50,000 SGD updates
CIFAR-10 CNN, augmented subsets	1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 64	44	0.96	50,000 SGD updates
Total	One seed per cell	229	14.98	Up to four runs/GPU

Table 3. Completed sweeps, exact model widths, and measured UCloud job durations. Each width is run at label-noise probabilities 0, 10, and 20%, except the clean CIFAR-100 CNN sweep (0% only) and the subset sweep. For the subset sweep, p is the label-noise probability (10 or 20%) and n is the number of training examples (12,500 or 25,000): 11 widths $\times$ 2 noise rates $\times$ 2 subset sizes yield 44 runs. Other runs use all 50,000 training examples. Durations are allocation wall times, rounded independently; overlapping model durations are not added to estimate GPU-hours.

Table 4: Coverage of the main paper's figures. Figure references point to our appendix comparisons. The final column describes differences or omitted conditions, rather than additional planned experiments.

Paper figure	Plot and coverage	Differences from the full reproduction
1–2	Figures 3 and 4: 10%-noise ResNet18 width sweeps and heatmaps through 400 epochs.	Original opening figures use 15% noise and 4,000 epochs; our grid is smaller and uses one seed. Figure 1 averages five trials.
3	Figure 5: IWSLT'14 Transformer model-size sweep, 25 embedding widths from 8 to 200, with 4,000 and 18,000 pairs.	Same selected widths, sample sizes and 80,000-update horizon; exact preprocessing, subsets and training configuration could not be recovered.
4	Figure 6: CIFAR-10 and CIFAR-100 ResNet18 at 0/10/20% noise, ten widths, epoch 400.	Smaller width grid, no CIFAR-10 5/15% conditions, and 400 rather than 4,000 epochs.
5	Figure 7: augmented and unaugmented CIFAR-10 CNN at 0/10/20% noise, through width 64, after 50,000 updates.	Smaller grid and 50,000 rather than 500,000 updates. Authors' saved measurement indices do not establish an exact time alignment.

Continued on the next page

Table 4 (continued)

Paper figure	Plot and coverage	Differences from the full reproduction
6	Figure 8: clean, unaugmented CIFAR-10 CNN with SGD and Adam, eleven widths.	One seed; 50,000 SGD updates or 400 Adam epochs rather than 500,000 updates or 4,000 epochs.
7	Figure 9: clean CIFAR-100 CNN at eleven shared widths.	One seed and 50,000 updates rather than five seeds, 64 widths and 1,000,000 updates.
8	Outside the reproduced subset: full-data IWSLT'14 and WMT'14 model-size sweeps.	Our translation experiments instead cover the smaller IWSLT'14 subsets in Figures 3 and 11(b).
9	Figure 10: three ResNet18 width trajectories and a ten-width heatmap through epoch 400.	Smaller width grid and 400 rather than 4,000 epochs.
10(c)	Figure 11: width-128 CIFAR-10 CNN trajectories at 0/20% noise, through 50,000 updates.	ResNet18 panels are outside our coverage. CNN horizon is reduced from 500,000 updates; authors' time coordinates are saved measurement indices.
11(a)	Figure 12: 20%-noise CIFAR-10 CNN at 12,500/25,000/50,000 examples and eleven common widths.	One seed and 50,000 updates; published panel averages five trials. Our additional 10%-noise subset runs lack a matching released source.
11(b)	Figure 13: IWSLT'14 sample-size sweep from 2,000 to 50,000 pairs at embedding widths 64 and 80.	80,000 updates; exact subsets, preprocessing and batching cannot be matched to the authors' configuration.
12	Figure 14: 20%-noise CNN heatmaps and slices at three sample sizes and eleven common widths.	Three rather than nine sample sizes, eleven rather than 42 widths, and 50,000 rather than 500,000 updates.

B. COMPARISON OF EXPERIMENTAL SETTINGS

Table 5 compares the completed vision experiments one setting at a time. Page numbers refer to the 2021 journal article (32 pages), not an earlier arXiv version [1]. Released code paths refer to commit `0d653296`. The release contains model definitions, plotting notebooks, and a `README.md` linking result archives; it does not provide the vision training loop or data loader. Where neither the paper nor those files specify a detail, we identify it as unstated rather than infer a setting from the results.

Table 5: Our model, data, and training settings, their source locations, and differences from the stated protocol.

Setting	Our setup	Paper setting and source location	Difference
Model			
Vision architectures	Released ResNet18 and five-layer CNN; widths $[k, 2k, 4k, 8k]$	Same architectures: p. 17, Appendix B.1. Code: `models/resnet18k.py` and `models/mcnn.py`.	Same definitions
Initialization	Random, default PyTorch initialization	Default PyTorch initialization: p. 17, Appendix B.2, "Initialization". Model files above instantiate PyTorch layers.	Same rule
Width grids	Widths listed in Table 3	Width sweeps: p. 17, Appendix B.1; exact arrays in linked archives, e.g. `cifar10-resnet18k-50k-adam/ks` and `pct-cifar10-mcnn-50000-p0-sgd-big/ks`. `README.md` supplies archive links.	Fewer widths
Data			
Pixel preprocessing	Divide by 255; no channel normalization	Not stated in Section 4 (p. 8) or Appendix B.2 (pp. 17–18). No released data-loader file.	Not stated
Label corruption	Fixed Bernoulli mask; uniformly incorrect labels; fixed across augmentations	Once-sampled, uniformly incorrect labels: p. 8, Section 4, "Label noise". Fixed labels across augmentations: p. 17, Appendix B.2. No released implementation.	Same rule; seed not stated
Noise settings	0, 10, 20%; Figure 1 compares 10% on both sides	Figure 4, p. 7, and experiment list in `README.md`; original Figure 1, p. 4, uses 15%.	Same selected rates; original Figure 1 differs
Augmentation	Padding 4, random crop 32, horizontal flip	RandomCrop(32, padding=4), RandomHorizontalFlip: p. 17, Appendix B.2, "Data-augmentation". No released loader.	Same operations
Subsampling	Seeded permutation; 12,500/25,000/50,000 training images	Sample sizes: Figure 11(a), p. 13. Random subsampling: p. 19, Appendix B.4, "Sample-wise nonmonotonicity". No released subsampling code.	Same sizes; permutation and seed not stated
Test evaluation	Clean test split; Figure 1 also transforms error to the expected noisy-test distribution	Clean test error except Figure 1: p. 8, Section 4. Transformation implemented in `intro_resnet_plot.ipynb`, "Simple Plot" and `plot`.	Same error convention
Train evaluation	Fixed training labels, without augmentation	Exact train-evaluation transform not stated in Appendix B.2 (pp. 17–18). No released evaluation loop.	Transform not stated
Training			
Loss	Cross-entropy	Cross-entropy: p. 17, Appendix B.2, "Loss function". No released training loop.	Same loss

Continued on the next page

Table 5 (continued)

Setting	Our setup	Paper setting and source location	Difference
Batch size	128	128: p. 18, Appendix B.2, "Batch size". No released loader.	Same size
Adam	Constant LR 10^{-4}; PyTorch defaults	Same settings: p. 17, Appendix B.2, "Optimization: Adam". No released optimizer code.	Same settings
SGD	LR 0.1, inverse-square-root; 512-step updates; momentum 0	Same prescription: p. 18, Appendix B.2, "SGD" and "Learning rate schedule descriptions". No released optimizer code.	Same prescription
Regularization	No weight decay or dropout	None unless stated otherwise: p. 17, Appendix B.2, "Regularization".	Same setting
Training horizon	Adam 400 epochs; SGD 50,000 updates	Adam 4,000 epochs; SGD 500,000 steps: p. 8, Section 4. CIFAR-100 CNN uses 1 million steps: Figure 20, p. 25, Appendix E.2.1.	10% of stated horizon; 5% for CIFAR-100 CNN
Trials	One seed (0) per condition	Five trials for Figures 1, 7 and 11(a), bottom panel: p. 19, Appendix B.4. Figure 1 averaging is in `intro_resnet_plot.ipynb`.	One trial rather than five
Precision	FP32	Arithmetic precision not stated in Section 4 or Appendix B.2. No released training loop.	Not stated
Evaluation cadence	Adam: 10 epochs; SGD: 1,250 updates	`intro_ocean_dynamics.ipynb` labels ResNet indices plus one as epochs. CNN logging interval is not established by the paper or release.	No confirmed common cadence
Run infrastructure	Our loop and checkpoints; up to four runs per B200	Checkpointing, concurrency and GPU scheduling not stated in Appendix B.2 or provided in the released files.	Our implementation

C. REPLICATION FIGURES: OURS COMPARED WITH THE AUTHORS

This gallery places our vision replication results alongside measurements reconstructed from the authors' released data, and our translation results alongside the published comparisons. Each caption identifies the shared settings and any differences in training horizon, trial count, or time coordinates. The Figure 1 comparison also appears in the main text. Released-data reconstructions are distinct from the original published artwork and are not additional experiments by our group [1].

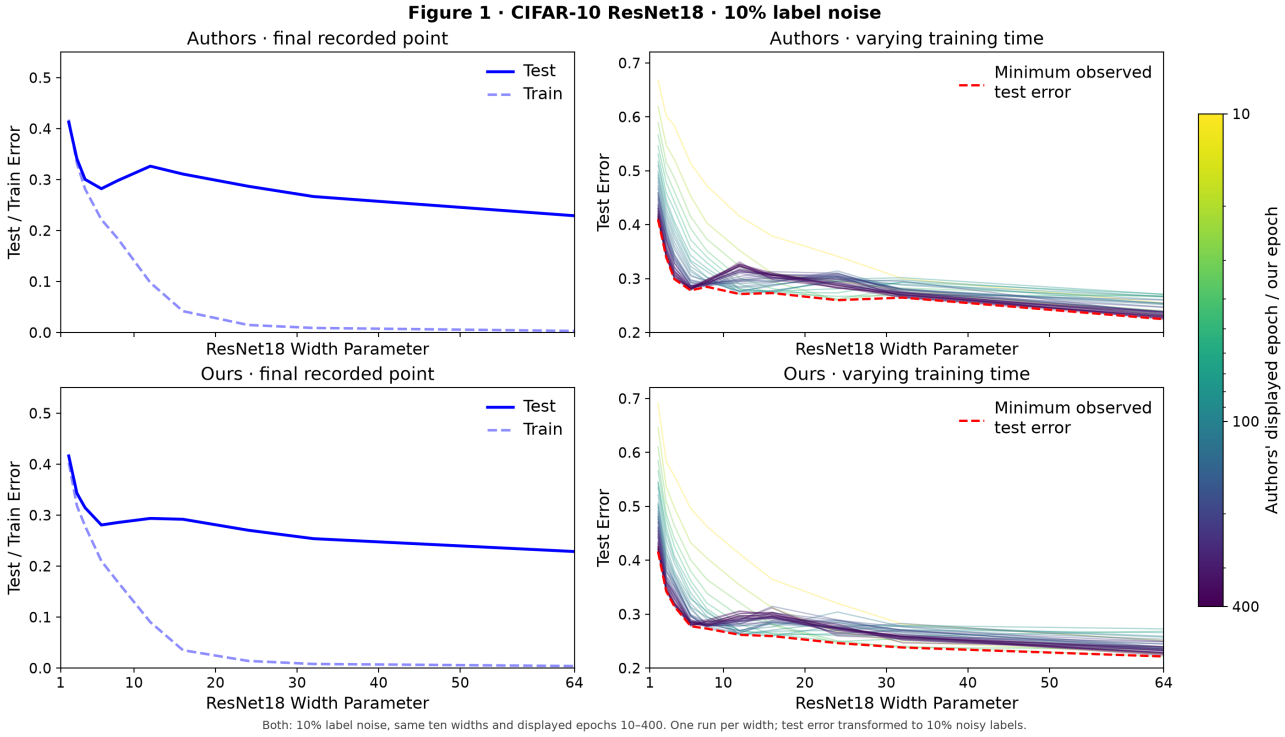


Fig. 3. CIFAR-10 ResNet18 width sweeps (paper Figure 1). Both authors and ours use 10%-noise CIFAR-10 ResNet18, the same ten widths and 400 displayed epochs, with one run per width. Authors' epochs follow the notebook convention; ours are recorded epochs. Test error uses the same expected noisy-test transformation.

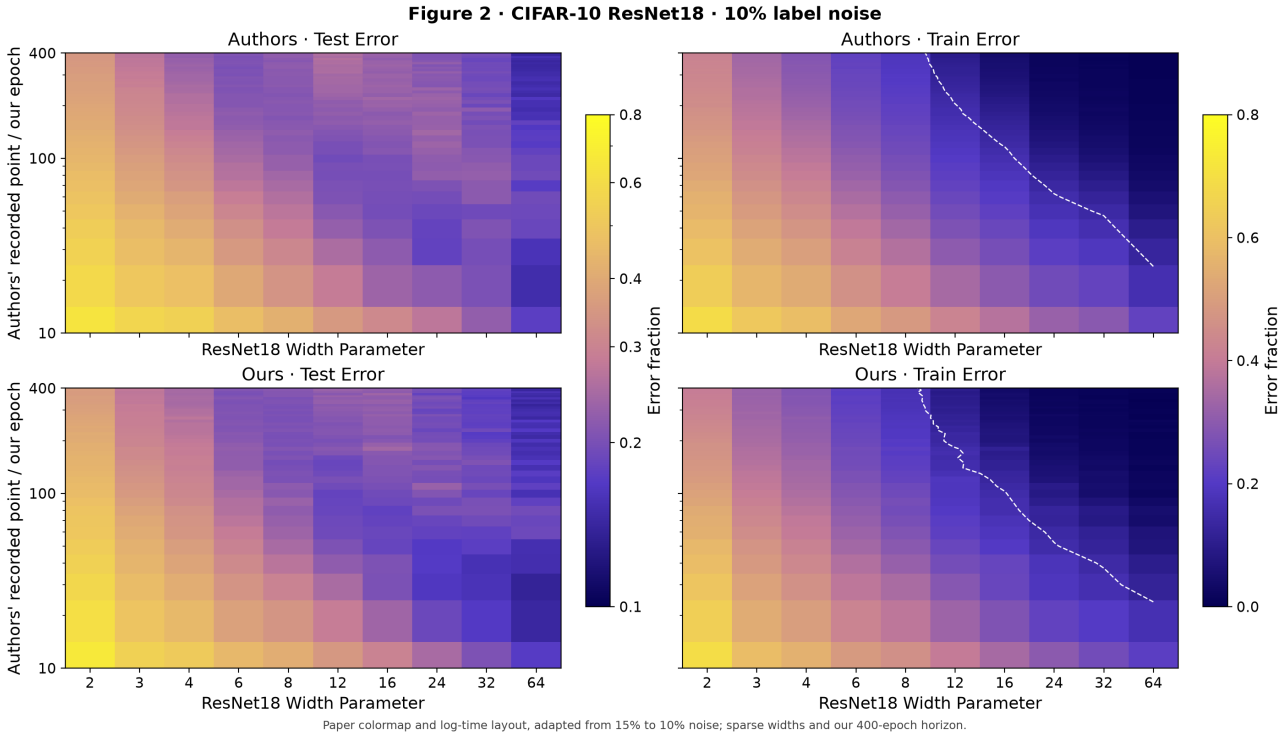
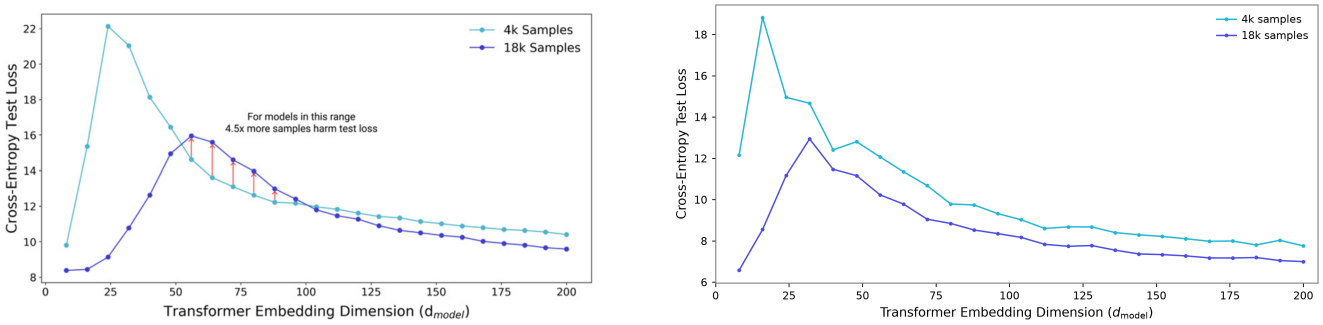


Fig. 4. Train and test heatmaps (paper Figure 2): authors' and our CIFAR-10 ResNet18 train/test heatmaps at 10% noise, on the same ten widths and displayed horizon. Authors' epoch coordinates follow the released notebook convention; ours are recorded.



(a) Nakkiran et al.

(b) Our reproduction

Fig. 5. Comparison of the published Figure 3 and reproduced model-wise translation experiment on IWSLT'14.

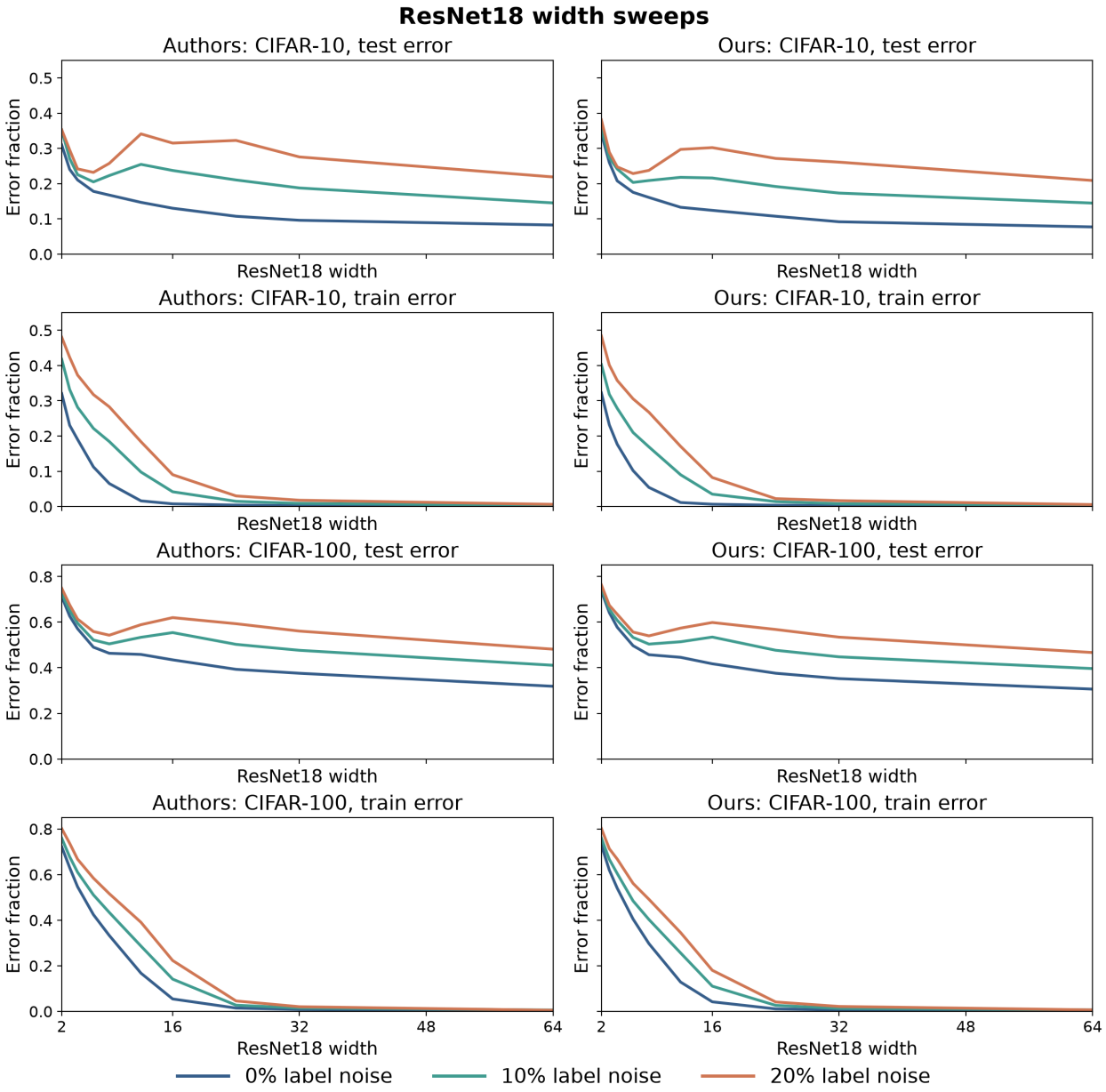


Fig. 6. ResNet18 error versus width on CIFAR-10 and CIFAR-100, reproducing paper Figure 4. Both sources use the same ten widths and label-noise rates of 0, 10, and 20%. The left column shows the authors at displayed epoch 400 (measurement index 399); the right column shows ours at recorded epoch 400.

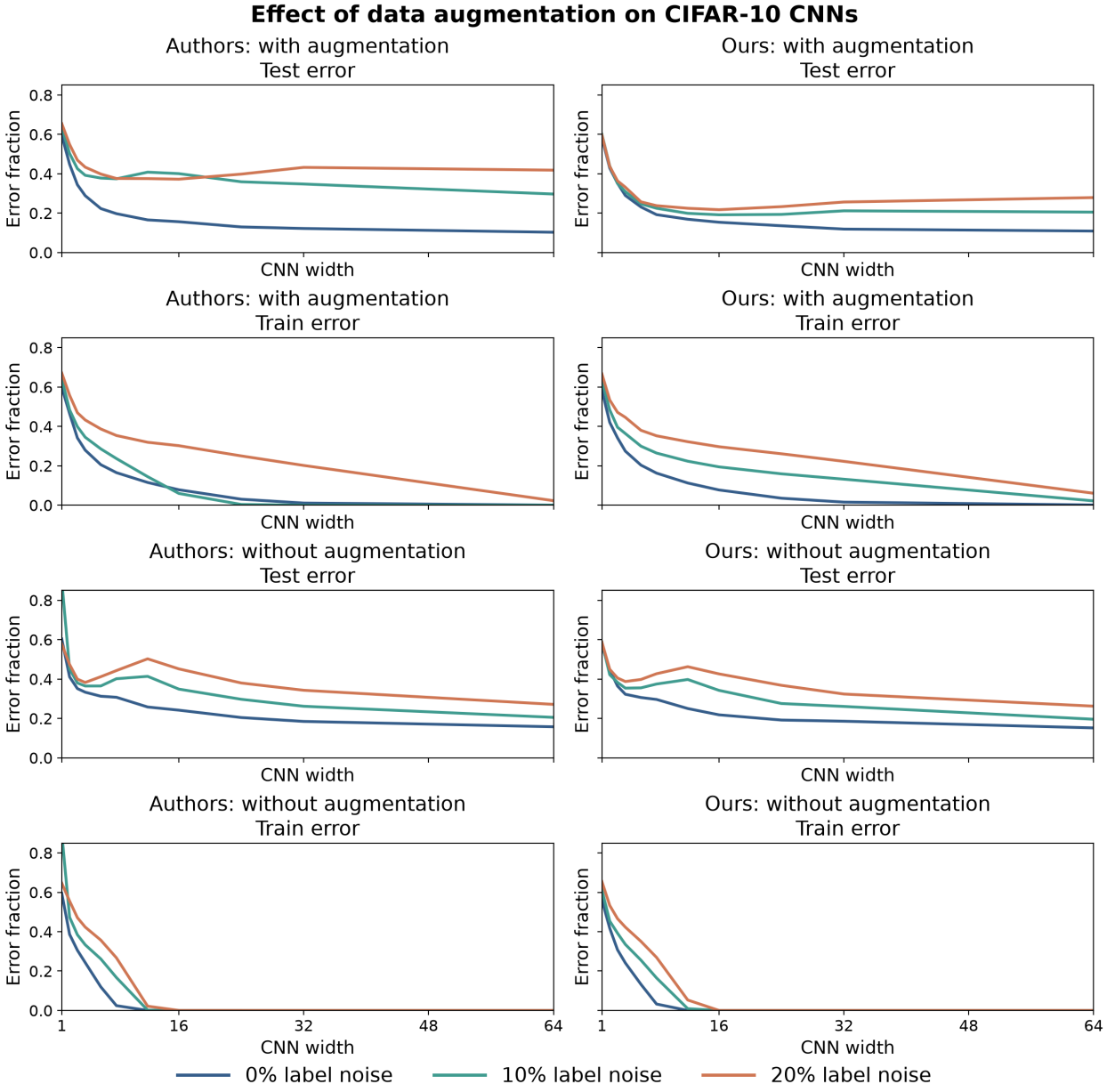


Fig. 7. Effect of data augmentation on CIFAR-10 CNN error, reproducing paper Figure 5. Both sources use the same eleven widths and label-noise rates of 0, 10, and 20%. The left column shows the authors at measurement index 194; the right column shows ours after 50,000 SGD updates. The author point corresponds to approximately 49,920 updates only under the unverified assumption of 256 updates per measurement, so training duration is not confirmed to match.

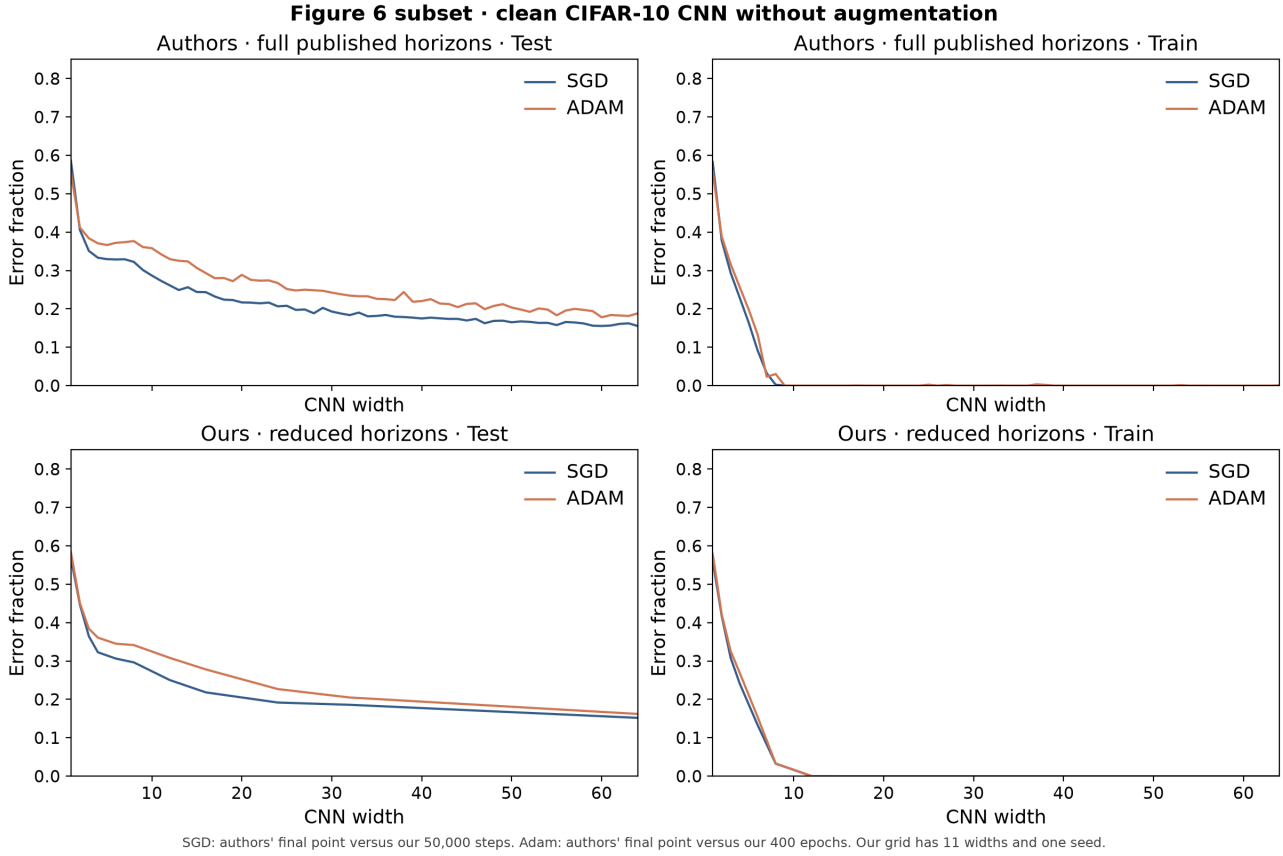


Fig. 8. SGD and Adam comparison (paper Figure 6): authors' and our clean, unaugmented CIFAR-10 CNN results with SGD and Adam. Authors' endpoints use full released histories; ours use 50,000 SGD updates or 400 Adam epochs.

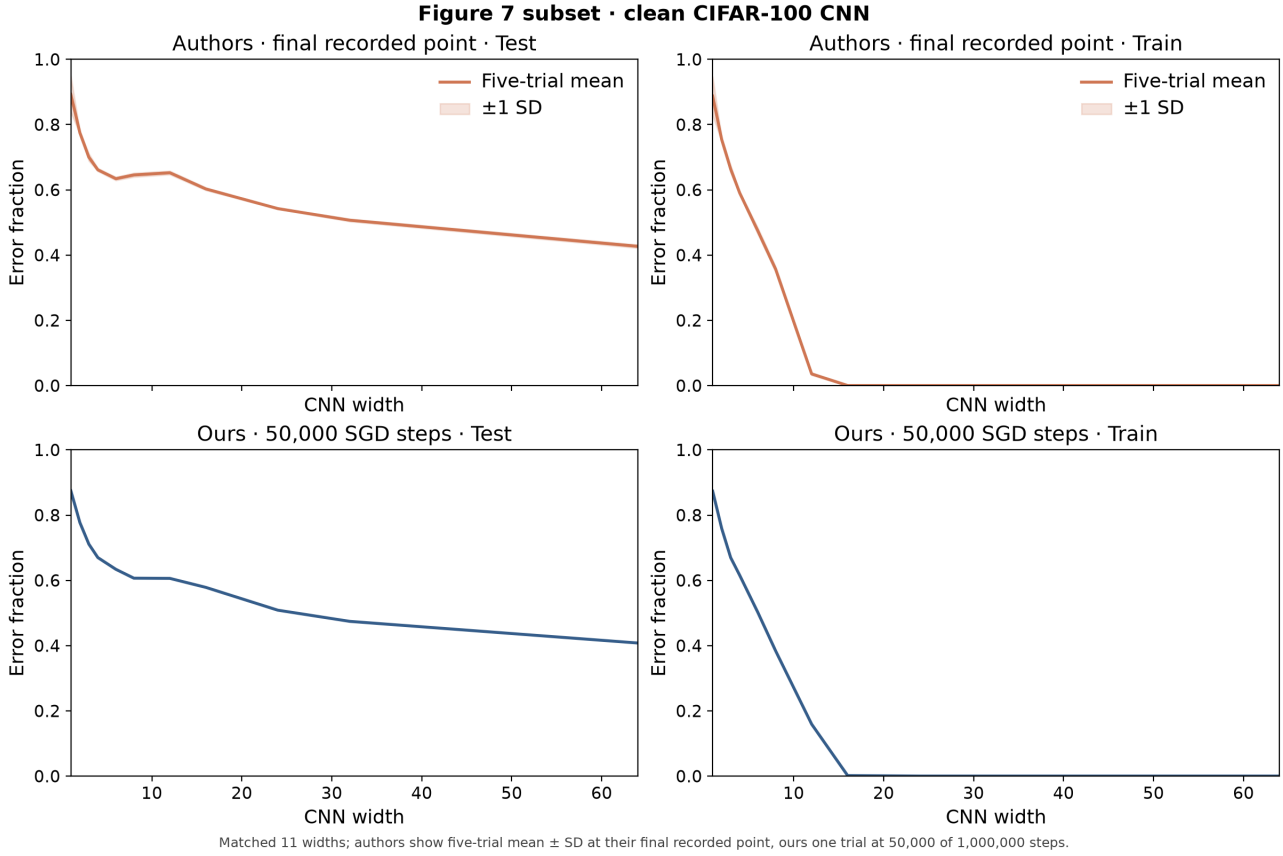


Fig. 9. CIFAR-100 CNN width sweeps (paper Figure 7): authors' and our clean CIFAR-100 CNN results at eleven common widths. The authors' five-trial mean and standard deviation use full histories; ours use one seed and 50,000 SGD updates.

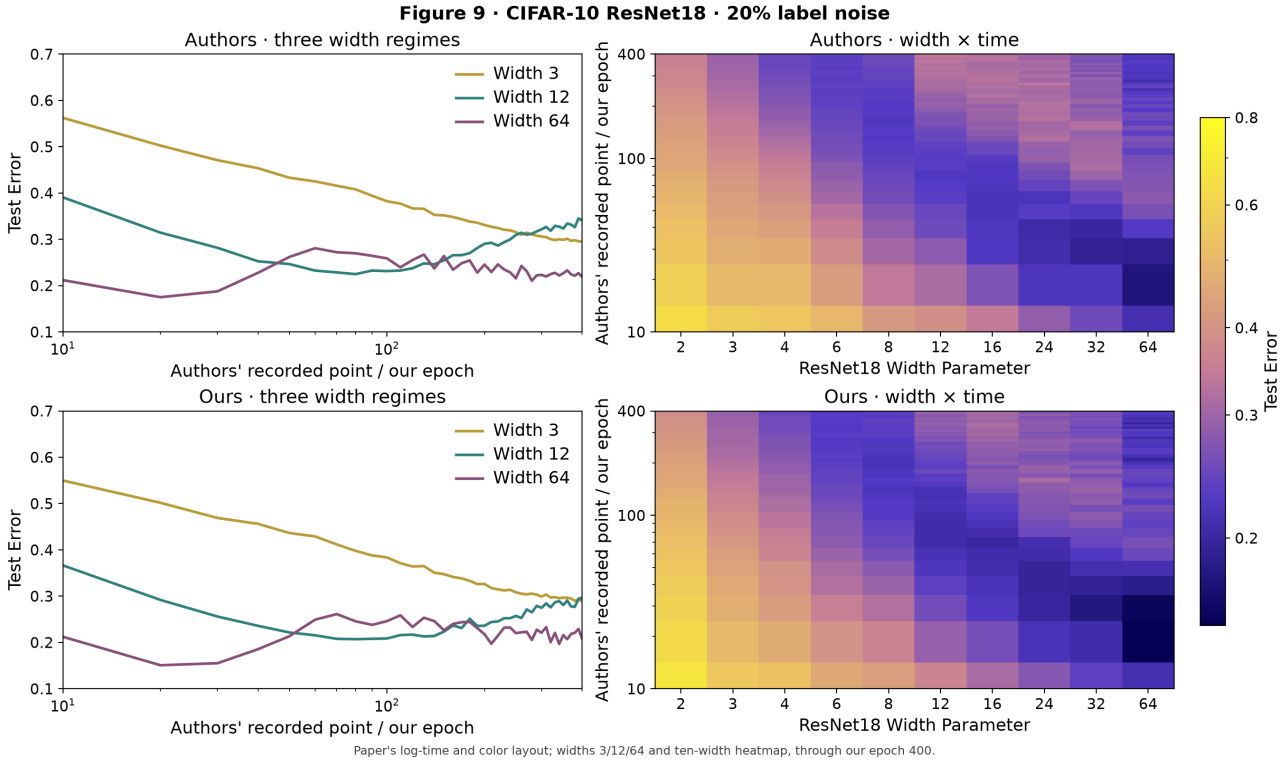


Fig. 10. Epoch-wise CIFAR-10 ResNet18 comparisons (paper Figure 9): three width trajectories and a ten-width heatmap through epoch 400, compared with the authors' released measurements. Our recorded epochs and the authors' notebook epoch convention are distinguished in the plots.

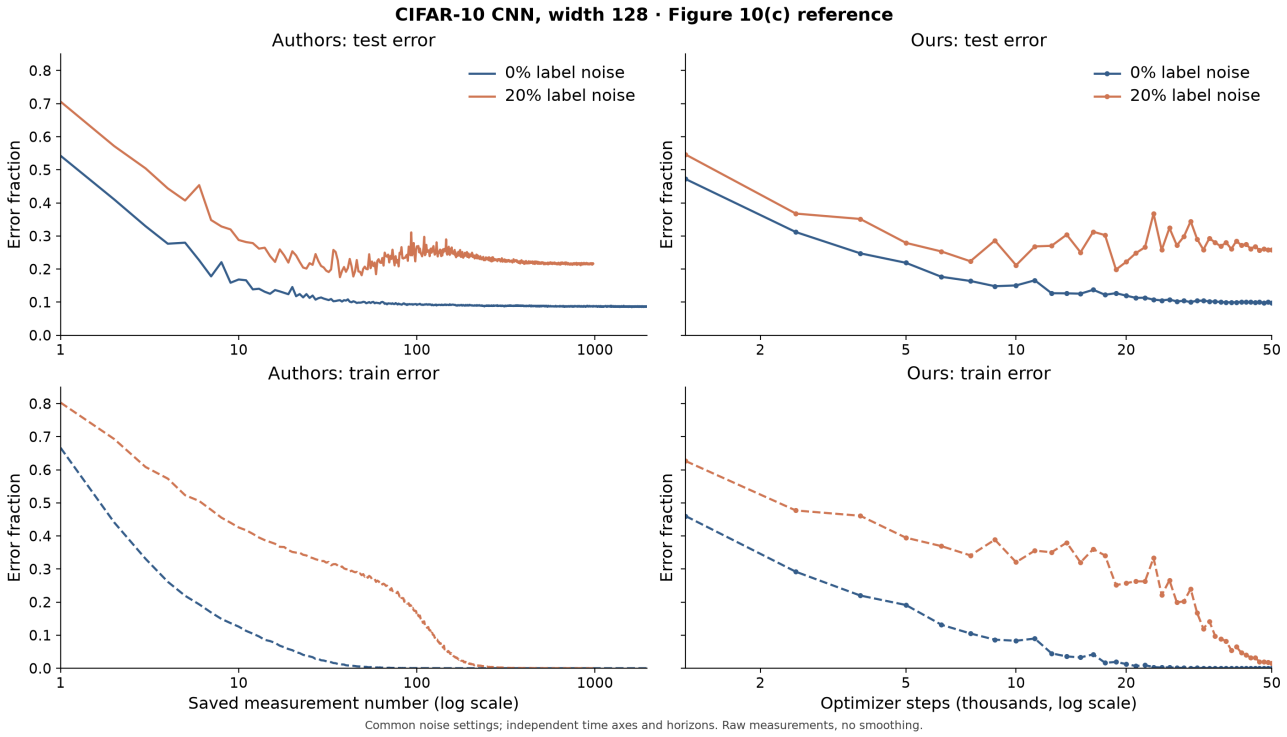


Fig. 11. Training trajectories (paper Figure 10(c)): authors' and our width-128 CIFAR-10 CNN trajectories at the common 0 and 20% noise settings. Authors' clean full-data and 50,000-example/20%-noise references use saved measurement numbers; ours use recorded optimizer steps through 50,000 updates. Logging intervals and horizons are not matched. No smoothing is applied. The unresolved 10%-noise author source is excluded.

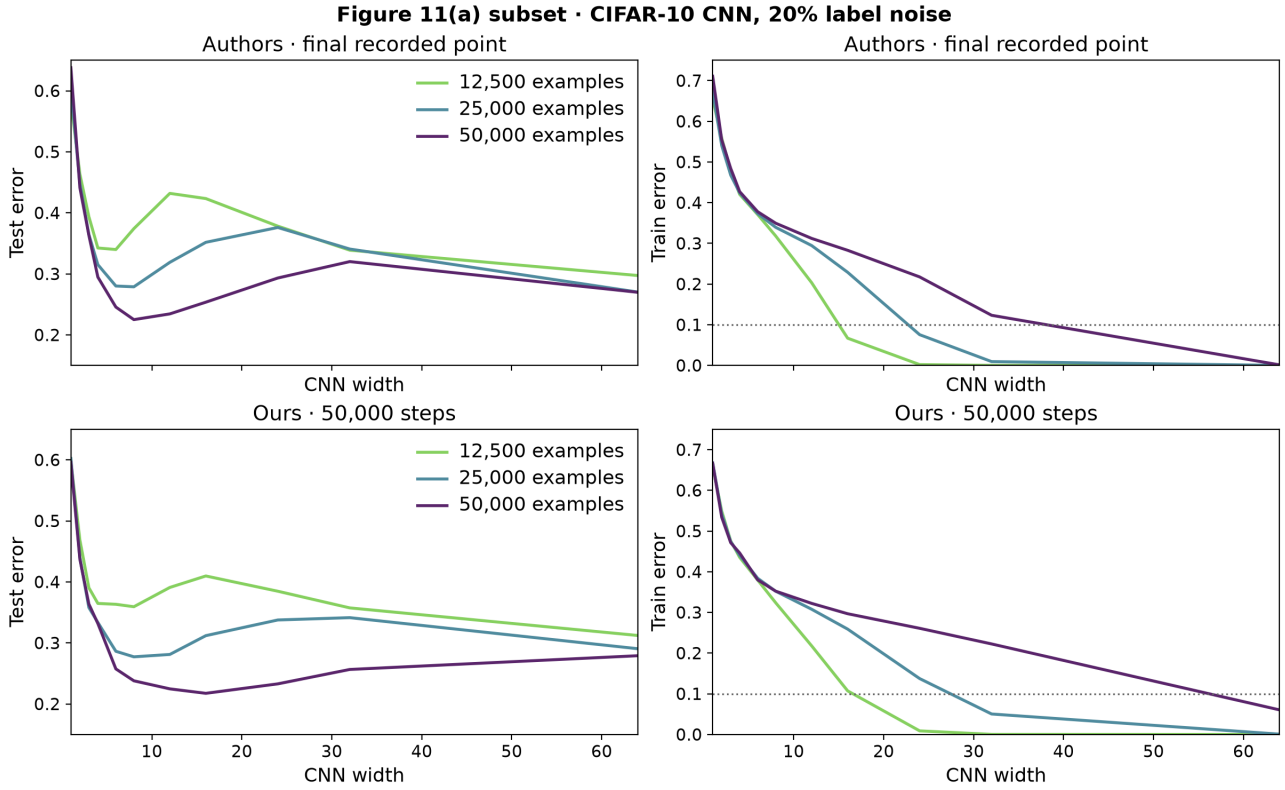
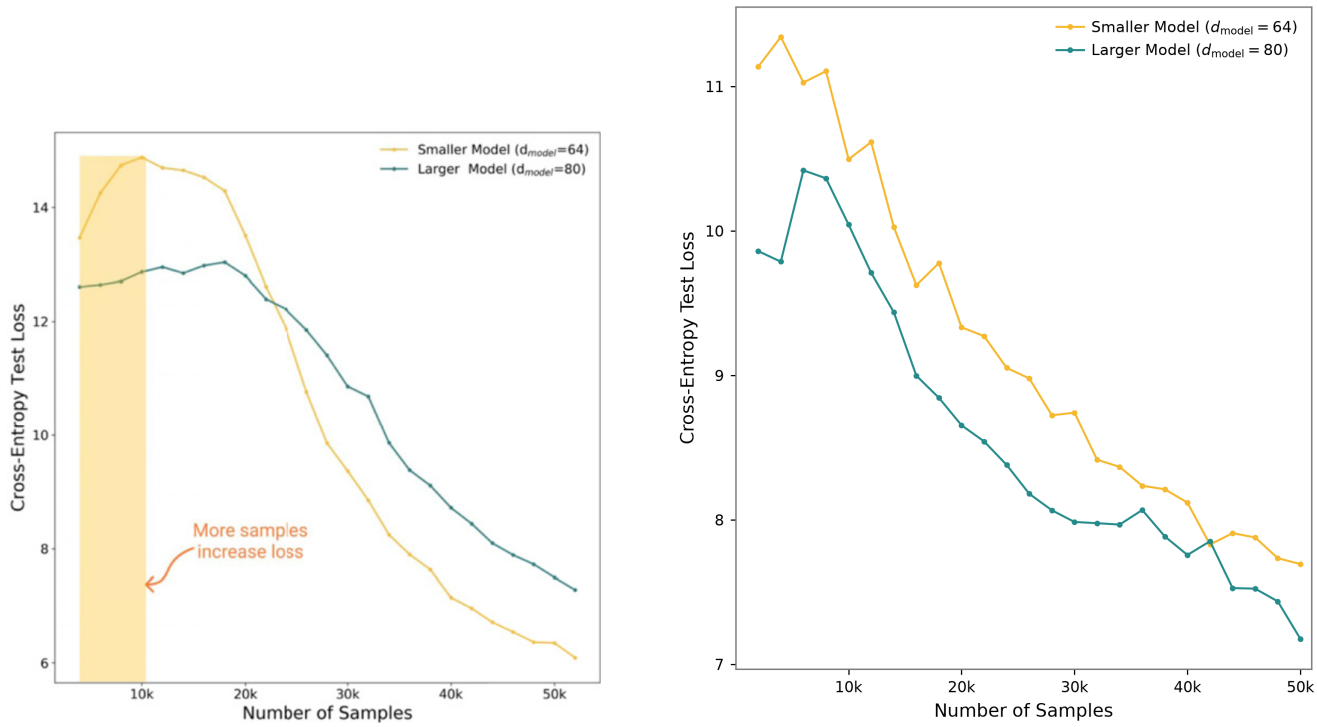


Fig. 12. Effect of training-set size (paper Figure 11(a), 20% panel): authors' and our 20%-noise CIFAR-10 CNN test and train error for 12,500, 25,000 and 50,000 examples at the eleven common widths. Authors' curves come from the released Figure 12 grid, a single trial, whereas the published panel shows the mean and standard deviation of five; ours are one seed at 50,000 updates. Dotted line: train error 0.1. Table 2 uses the finer grid of each side. Our 10%-noise subset runs are excluded because no matching author source was identified.



(a) Nakkiran et al.

(b) Our reproduction

Fig. 13. Comparison of the published Figure 11(b) and reproduced sample-wise translation experiment on IWSLT'14.

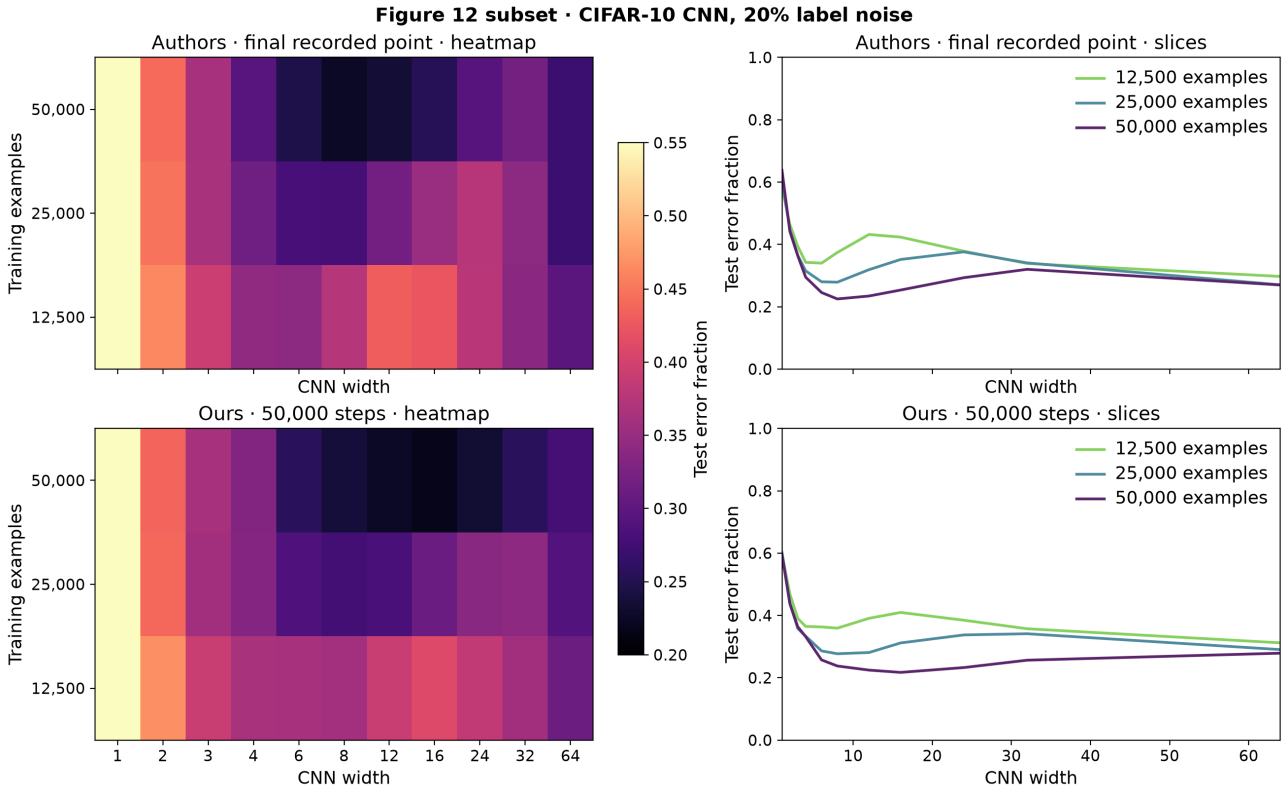


Fig. 14. Sample-size heatmaps and slices (paper Figure 12): authors' and our 20%-noise CIFAR-10 CNN sample-size heatmaps and slices. Both use three sample sizes, eleven common widths, and a common color scale; training horizons differ.